

Gelişmiş PE Analizi, Adli Bilişim Standartları ve Heuristik Skorlama Algoritmaları Üzerine Derinlemesine Araştırma Raporu

Modern siber tehdit ekosistemi, statik imza tabanlı tespit sistemlerini atlatmak amacıyla tasarlanmış, matematiksel olarak optimize edilmiş ve bellek içi (in-memory) çalışma prensiplerine dayanan karmaşık mimarilere evrilmiştir. Windows Taşınabilir Çalıştırılabilir (Portable Executable - PE) dosyalarının statik ve dinamik analizi, tehdit istihbaratı ile entegre edildiğinde devasa veri kümeleri üretir. Ancak bu verilerin anlamlandırılması; zararlı yazılımların kod karmaşıklığı algoritmalarını nasıl manipüle ettiğinin anlaşılmasını, toplanan bulguların dijital delil zinciri (Chain of Custody) standartlarına uygun olarak hukuki çerçevede nasıl korunacağını bilmesini ve bu göstergelerin matematiksel bir heuristik risk skorlama motorunda nasıl ağırlıklandırılacağını tasarlanmasını gerektirir.

Aşağıdaki çok boyutlu araştırma raporu, yapısal dosya incelemesinden hukuki kabul edilebilirliğe ve makine öğrenmesi destekli karar algoritmalarına uzanan 50 adımlık ardışık bir çözümleme felsefesi üzerine inşa edilmiştir. Bu analiz, yüzeysel tanımlamalardan tamamen arındırılmış olup, doğrudan bellek haritalama (Memory Mapping), x86/x64 Assembly yönergeleri, Win32 API manipülasyonları ve ISO/IEC 27037 Adli Bilişim standartları ekseninde sunulmaktadır.

Bölüm I: İleri Seviye Gizlenme (Obfuscation) Mimarisi ve Statik Analizin Sınırları

Statik analiz yaklaşımlarında karşılaşılan en büyük handikap, hedef dosyanın diske yazılı hali ile bellekteki (RAM) çalışma zamanı (runtime) halinin birbirinden tamamen farklı olabilmesidir.

Geleneksel yaklaşımlar, PE başlıklarındaki anormallikleri ve kod bölümlerinin entropisini (karmaşıklığını) baz alarak zararlı yazılım tespiti yapmaya çalışır. Ancak sofistike saldırı vektörleri, bu ölçümleri sistematik olarak manipüle eder.

Adım 1: Shannon Entropisi Paradigmasının Çöküşü ve Empirik Yanılgılar Zararlı yazılım analizi literatüründe uzun yıllar boyunca kabul gören en büyük yanılgılardan biri, düşük entropi skorunun (genellikle 7.0'ın altı) bir dosyanın paketlenmemiş (non-packed) veya şifrelenmemiş (non-obfuscated) olduğu anlamına geldiği varsayımdır. Yapılan geniş çaplı ampirik araştırmalar, düşük entropiye sahip 50.000 Windows zararlı yazılım örneğinden oluşan bir veri setinin incelenmesi sonucunda, bu örneklerin %30'undan fazlasının aslında çalışma zamanlı (runtime) paketleme şemaları kullandığını kesin olarak kanıtlamıştır.¹ Bu durum, Shannon entropisini yalıtılmış bir metrik olarak kullanan analiz motorlarının temelden kusurlu kararlar üretebileceğini göstermektedir. Geliştirilen bir PE analiz aracı, pefile kütüphanesi aracılığıyla elde ettiği 7.0 üzeri entropi skorlarında UPX veya Themida gibi ticari paketleyicileri kolayca tespit edebilse de, düşük entropili polimorfik paketleyiciler karşısında kör (blind-spot) kalmaktadır.

Adım 2: Düşük Entropili Paketleme (Low-Entropy Packing) Mekanizmaları Gelişmiş saldırırganlar, Shannon Entropisi skorunu bilinçli olarak düşürmek veya meşru yazılım aralıklarında (örneğin 4.5 ile 5.5 arası) sabit tutmak için çeşitli yapısal manipölasyon teknikleri kullanırlar. Bayt doldurma (byte padding), kodlama (encoding) manipölasyonları ve tek alfabeli yer değıştirme (monoalphabetic substitution) gibi teknikler, veri bloğunun rastgeleliğini (randomness) azaltarak entropi hesaplama algoritmalarını yanıltır.³ Örneğin, şifrelenmiş (encrypted) bir veri bloğunun sonuna yüksek oranda tekrarlayan Null (0x00) baytları veya NOP (0x90) kaydırıcıları eklenmesi, toplam bölüm (section) entropisini matematiksel olarak aşağı çeker. Bu bağlamda, destek vektör makineleri (SVM) veya rastgele orman (RF) gibi makine öğrenmesi sınıflandırıcıları dahi düşük entropili paketlenmiş zararlı yazılımları tespit etmekte yetersiz kalmaktadır.¹

Adım 3: Polimorfik ve Metamorfik Motorların Evrimi Polimorfik motorlar, imza tabanlı tespitleri (signature-based detection) atlatmak için deşifre edici (decryptor) kod bloklarını her bulaşma (infection) döngüsünde yeniden üretir.⁴ "The Mutation Engine" (MtE) gibi erken dönem örneklerden modern varyantlara kadar uzanan bu yapılar, yazılımın gövdesini şifreler ve sadece çalışma zamanında deşifre eder. Metamorfik motorlar ise mimariyi bir adım öteye taşıyarak yazılımın kendi gövdesini enfeksiyon aşamasında tamamen yeniden yazar; bunu yaparken yazmaç takası (register swapping), ölü kod ekleme (dead-code insertion) ve talimat ikamesi (instruction substitution) gibi teknikler kullanarak her iterasyonda tamamen farklı bir derleme (assembly) matrisi üretir, böylece hem entropiyi değıştirir hem de sabit disk imzalarını tamamen yok eder.⁴

Adım 4: "Pseudo-Instruction" Paradigması ve Derin Öğrenme Modelleri Entropi analizinin yetersiz kaldığı bu noktada, dilbilimsel yaklaşımlardan ilham alan yeni nesil derin öğrenme modelleri devreye girmektedir. Pack-ALM gibi Assembly Dil Modelleri (Assembly Language Models), paketleyici tespitini entropi ölçümünden çıkarıp "gerçek" ve "sözde" (pseudo) komutları ayırt etme görevine dönüştürmüştür.⁶ Bu mimariler, yerel ve paketlenmiş verileri ön işleyerek, paketlenmiş verilere özgü komut dizilimi anormalliklerini doğal dil işleme mantığı ile yakalar. Gelişmiş PE analizörleri için statik entropi hesabının ötesine geçerek komut seti yapısını (instruction format) inceleyen bu tür n-gram tabanlı veya transformer tabanlı çıkarım motorlarının entegrasyonu zorunluluk haline gelmektedir.⁶

Adım 5: Bellek İçi (In-Memory) Kaçınma ve İşlem İçi Manipölasyon Disk üzerindeki izlerini (footprint) minimize eden modern tehditler, Process Hollowing, Reflective DLL Loading veya doğrudan Fileless (Dosyasız) saldırı vektörlerini benimser.⁷ Bu teknikler, zararlı kodun disk üzerinde şifreli (veya yapısal olarak zararsız görünümde) kalmasını sağlarken, meşru bir işlem (örneğin svchost.exe) askıya alınarak (suspended state) belleğe yüklendiğinde kendi deşifre edilmiş zararlı yükünü bu meşru işlemin bellek alanına enjekte eder.⁷ Bu yaklaşım, statik analiz araçlarının diskteki PE dosyasının bölümlerini ayrıştırmakla elde ettiği verileri geçersiz kılar, çünkü gerçek çalışma mimarisi yalnızca bellek eşlemesinde (memory mapping) görünür durumdadır.

Evasion (Atlatma) Tekniğı	Statik Analize Etkisi	Bellek (Memory) Davranışı	Geleneksel Tespit Yöntemi
---------------------------	-----------------------	---------------------------	---------------------------

High-Entropy Packing	Kod bölümleri okunamaz, entropi > 7.0	Çalışma zamanında bellek tahsisi ve deşifre	Heuristik entropi analizi
Low-Entropy Packing	Entropi 4.5-5.5 arası, meşru PE gibi görünür	Özel şemalarla kademeli bellek açma	ALM (Assembly Language Models)
Polymorphism	Sadece decryptor stub değişir	Gövde bellekte hep aynı şekilde açılır	Sandbox emulation, memory dumping
Metamorphism	Tüm gövde assembly kodları değişir	Bellekte de farklı opcode matrisleri üretir	Behavioral analysis, API hooking

Bölüm II: Win32 API Mimarisi, IAT Atlatma ve Dinamik Çözümleme

Zararlı yazılımların EDR (Endpoint Detection and Response) sistemlerinden ve statik PE analizörlerinden kaçınmak için başvurduğu en yaygın ve karmaşık tekniklerden biri, Windows API çağrılarının Import Address Table (IAT) dışına çıkarılmasıdır.

Adım 6: Import Address Table (IAT) Bağımlılığının Ortadan Kaldırılması Geliştirilen araç, pefile kütüphanesi kullanarak Windows PE başlıklarını ve IAT yapısını başarılı bir şekilde ayrıştırabilir. IAT, bir programın çalışması için işletim sisteminden talep ettiği dışa aktarılmış fonksiyonların (exported functions) listesini barındırır. Ancak sofistike zararlı yazılımlar (TrickBot, QakBot vb.), VirtualAlloc, CreateRemoteThread veya WriteProcessMemory gibi şüpheli ve agresif API'leri IAT içerisine hiçbir zaman dahil etmez.⁸ Analistler tarafından tespit edilmemek adına, zararlı yazılım bu bağımlılıkları çalışma zamanında kendi imkanları ile çözer.

Adım 7: API Hashing Tekniğinin Anatomisi ve İmzaların Gizlenmesi Saldırganlar, Windows API fonksiyonlarının düz metin (plaintext) isimlerini ("LoadLibraryA" gibi) dosya içerisine string (dize) olarak gömmek yerine, bu isimleri geliştirme aşamasında FNV-1A, CRC32 veya DJB2 gibi kriptografik olmayan özet algoritmalarıyla hashlerler.⁸ Örneğin, LoadLibraryA fonksiyonunun FNV-1A karşılığı olan 0x72F2B53C değeri ikili (binary) dosya içerisine gömülür. Çalışma zamanında ilgili string değerleri bellekte asla var olmadığı için, Regex temelli IoC avcıları veya strings gibi temel statik analiz komutları bu fonksiyon çağrılarını tespit edemez.⁸

Adım 8: Dinamik Çözümleme (Dynamic Resolution) ve GetProcAddress Çıkmazı Normal şartlarda bir program, IAT'de olmayan bir fonksiyonu yüklemek için Windows'un meşru LoadLibrary ve GetProcAddress fonksiyonlarını kullanır.¹⁰ Ancak bu iki API'nin yoğun ve şüpheli kullanımı EDR sistemleri tarafından sıkı bir şekilde izlenmektedir. Bu izlemeyi atlatmak isteyen

zararlı yazılımlar, GetProcAddress'i çağdırmaktan tamamen vazgeçerek, işletim sisteminin bağlayıcı (linker) ve yükleyici (loader) rollerini üstlenip hedef API'lerin adreslerini doğrudan bellekten manuel olarak çıkarırlar.⁹

Adım 9: Process Environment Block (PEB) Yürüyüşünün (Walk) Başlatılması Bir API'nin bellek adresini manuel olarak bulmanın ilk adımı, zararlı yazılımın kendi işlem ortam bloğuna (Process Environment Block - PEB) doğrudan erişmesidir. İşletim sistemi, çalışan her süreç (process) için bir PEB veri yapısı oluşturur. Zararlı yazılım, x86 mimarisinde `mov eax, fs:[30h]` ve x64 mimarisinde `mov rax, gs:[60h]` assembly komutlarını yürüterek doğrudan Thread Information Block (TIB) üzerinden PEB'in bellek taban adresini (base address) elde eder.⁹

Adım 10: PEB_LDR_DATA Yapısına Navigasyon PEB yapısına ulaşıldıktan sonra, zararlı yazılım belleğe halihazırda işletim sistemi tarafından yüklenmiş olan dinamik bağlantı kütüphanelerinin (DLL) haritasını çıkarmak zorundadır. Bunun için, PEB içerisindeki Ldr işaretçisi (pointer) kullanılarak PEB_LDR_DATA yapısına geçiş yapılır.⁹ Bu yapı, sürecin (process) yükleme sırası, bellek sırası ve ilklendirme sırasına göre düzenlenmiş modül listelerini içerir.

Yapı (Structure)	Offset (x64)	Açıklama
TEB	gs:	Thread Environment Block başlangıcı.
PEB Pointer	gs:[60h]	TEB içinden PEB'in adresine işaret eden offset.
Ldr (PEB_LDR_DATA)	``	PEB içinden yüklü modüllerin listesini barındıran yapıya offset.
InMemoryOrderModuleList	[Ldr + 20h]	Bellekteki modüllerin çift bağlı listesi (doubly linked list).
DllBase	``	Bulunan modülün bellekteki başlangıç adresi (Base Address).

Adım 11: Modül Listelerinde İterasyon (Linked List Traversal) PEB_LDR_DATA içerisindeki InLoadOrderModuleList veya InMemoryOrderModuleList çift bağlı listelerine ulaşıldığında, yazılım bir döngü başlatarak hafızadaki tüm LDR_DATA_TABLE_ENTRY bloklarını tek tek okur.⁹ Hedef, kernel32.dll veya ntdll.dll modüllerini bulmaktır. Bulunan her bir girişin

BaseDllName alanı incelenir (genellikle ismin kendisi de hashlenmiş olarak karşılaştırılır, böylece statik analizden tamamen kaçınılır).

Adım 12: Base Address Çıkarımı ve PE Başlıklarının Hafızadan Okunması kernel32.dll başarıyla bulunduğunda, bu yapının içinden modülün belleğe yüklendiği taban adresi (Base Address / DllBase) çekilir.⁹ Taban adresi, belleğe eşlenmiş bir PE dosyasının IMAGE_DOS_HEADER yapısının başlangıcını gösterir. Zararlı yazılım, diskteki dosya yerine bellekteki bu kernel32.dll yansıması üzerinde kendi "pefile" mantığını çalıştırarak e_lfanew üzerinden IMAGE_NT_HEADERS yapısına ulaşır.⁹

Adım 13: Export Address Table (EAT) Ayırıştırması NT Başlıkları içerisindeki OptionalHeader'da bulunan Data Directory dizisi okunarak IMAGE_DIRECTORY_ENTRY_EXPORT (Export Directory) tablosunun Relatif Sanal Adresi (RVA) elde edilir. Zararlı yazılım, bu RVA'yı Base Address ile toplayarak EAT'nin mutlak bellek adresini bulur. EAT içerisinde AddressOfNames (fonksiyon isimlerinin string pointer'ları), AddressOfNameOrdinals (isimlerin indeks sıra numaraları) ve AddressOfFunctions (gerçek fonksiyon adreslerinin RVA'ları) dizileri tespit edilir.⁹

Adım 14: Çalışma Zamanlı Dinamik Hashing ve Eşleştirme Zararlı yazılım, AddressOfNames dizisi üzerinde bir döngü başlatarak kernel32.dll'in dışa aktardığı tüm fonksiyon isimlerini (örneğin "VirtualAlloc") okur. Okunan her bir isim, çalışma zamanında (runtime) aynı algoritma (örn: FNV-1A) ile hashlenir. Ortaya çıkan hash değeri, yazılımın kendi içine gömdüğü hedef hash değeri (örneğin 0x72F2B53C) ile karşılaştırılır.⁹ Bu yöntem, hiçbir aşamada hedef API isminin bellekte tam olarak açık bir string gibi görünmemesini sağlar.

Adım 15: Mutlak Sanal Adresin (Absolute Virtual Address) Hesaplanması Hedef hash eşleştiğinde, fonksiyonun AddressOfNames dizisindeki indeks pozisyonu alınır. Bu indeks, AddressOfNameOrdinals dizisine verilerek fonksiyonun "sıra numarası" (ordinal) öğrenilir. Elde edilen sıra numarası AddressOfFunctions dizisinde bir indeks olarak kullanılarak fonksiyonun bellekteki RVA'sı çekilir. Nihayetinde, bu RVA değeri kernel32.dll'in Base Address'i ile toplanır.⁹ Ortaya çıkan mutlak sanal adres, LoadLibraryA veya GetProcAddress'in çağrılacağı doğrudan bellek işaretçisidir. Sistem bu şekilde statik araçları kör bırakarak kendi linker mekanizmasını inşa etmiş olur.

Bölüm III: EDR Kancalama (Hooking) Mekanizmaları ve Syscall (Sistem Çağrısı) İstismarları

Dinamik API çözümleme işlemi GetProcAddress gibi kullanıcı modu (user-mode) fonksiyonları statik izlemeden kaçırır da, gelişmiş EDR çözümleri API hooking (kancalama) teknikleri ile çalışma zamanı (runtime) tespiti yapmaktadır.

Adım 16: Kullanıcı Modu (Ring 3) EDR Kancalarının Mantığı Modern güvenlik ürünleri (EDR/XDR), işletim sistemi yüklenirken süreç alanına kendi izleme DLL'lerini (örneğin amsi.dll veya vendor spesifik DLL'ler) enjekte eder ve ntdll.dll içerisindeki NtAllocateVirtualMemory gibi kritik API'lerin başlangıç baytlarını bir JMP (jump) talimatı ile değiştirerek yürütme akışını (execution flow) kendi analiz motorlarına yönlendirir (Hooking).¹³ Kötü amaçlı bir bellek ayırma talebi tespit edilirse, süreç durdurulur.

Adım 17: Direct Syscalls (Doğrudan Sistem Çağrıları) Tekniği EDR'ların ntdll.dll üzerindeki

bu kancalarını aşmak için saldırganlar, Direct Syscalls (Doğrudan Sistem Çağrılarını) tekniğini geliştirmiştir. Windows mimarisinde bir API çağrısı, kullanıcı modundan çekirdek (kernel) moduna geçerken bir Sistem Servis Numarası (System Service Number - SSN) ile birlikte syscall assembly komutunu yürütür.¹³ Direct Syscalls yaklaşımında zararlı yazılım, ntdll.dll'i kullanmayı tamamen reddeder; kendi kod gövdesinde gerekli SSN numaralarını (örneğin 0x18) EAX yazmacına yükler ve doğrudan syscall komutunu kendi içinden tetikler.¹⁴ Bu durum, EDR'in Ring 3 seviyesindeki tüm kancalarını transit geçerek EDR'ı tamamen işlevsiz bırakır.

Adım 18: Indirect Syscalls (Dolaylı Sistem Çağrılarını) Tekniği Direct Syscalls tekniği güçlü olsa da, EDR'lar bu kez de "kendi modül alanı dışından (unbacked memory) gelen syscall komutlarını" tespit etmeye başlamıştır. Bu savunmayı aşmak için Indirect Syscalls geliştirilmiştir.¹³ Bu teknikte saldırgan, yine parametreleri kendi hazırlar, ancak syscall komutunu doğrudan çalıştırmak yerine, yürütme akışını ntdll.dll içerisindeki meşru syscall opcode'unun bulunduğu adrese (JMP) yönlendirir.¹⁶ Sistem, çağrının meşru bir Windows kütüphanesinden geldiğini sandığı için anomali tespiti başarısız olur.¹⁷

Adım 19: ntdll.dll Unhooking (Kancadan Kurtulma) Diğer bir atlatma tekniği ise, diske yazılı olan temiz ntdll.dll dosyasını C:\Windows\System32\ dizininden veya "KnownDll" ön belleğinden okuyarak, bellekte EDR tarafından modifiye edilmiş (kancalanmış) ntdll.dll bölümünün üzerine (Memory Mapping/Remapping) yazmaktır.¹⁶ Geliştirilen PE analizörünün statik motoru, IAT'de herhangi bir iz bırakmayan ve tüm yürütme akışını syscall manipülasyonu veya unhooking ile gerçekleştiren bu mimarileri salt PE dosya analizi ile tespit edemez; davranışsal kum havuzu (sandbox) telemetrisine¹⁷ entegre Ring 0 (Kernel) seviyesinde bir Call Stack izleme modülüne ihtiyaç duyar.

Bölüm IV: Olay Müdahalesi, Hukuki Geçerlilik ve Adli Bilişim (Forensics) Standartları

Teknik analiz boyutundan adli boyuta geçildiğinde, Gelişmiş PE Analiz ve Tehdit İstihbarat Aracının (PE Analyzer) ürettiği çıktıların kurumsal bir siber olay soruşturmasında (örneğin fidye yazılımı sızması) "Dijital Delil" olarak kabul edilebilmesi için katı hukuki standartlara uyulması zorunludur.

Adım 20: Dijital Delil Kabul Edilebilirliği (Admissibility) Hukuk mahkemelerinde veya ceza yargılamalarında bir dijital verinin delil olarak kabul edilebilmesi için mahkemeler temel olarak dört şartın yerine getirilmesini talep eder: Kaynağın doğrulanması (Authentication), bütünlüğün kanıtlanması (Integrity), delil zincirinin eksiksiz belgelenmesi (Chain of Custody) ve prosedürel uyumluluk (Procedural Compliance).¹⁸ Bu kritik gereksinimlerden sadece birindeki en ufak bir sapma (örneğin analiz raporunun zaman damgasının manipüle edilebilir olması), karşı taraf avukatlarının "delilin reddi" talebinde bulunmasına (Motion to Suppress) zemin hazırlar.¹⁸

Adli Bilişim Kriteri	Hukuki Karşılığı (ABD - Federal Rules / ISO)	Teknik Gereksinim	İhlal Sonucu

Authentication	Federal Rule 901/902	Delilin iddia edilen kaynaktan geldiğinin ispatı.	Kanıt mahkemede sunulamaz.
Integrity	ISO/IEC 27037 / SHA-256	Toplandığı andan itibaren bit seviyesinde değişmezlik.	Karşı itirazla delil çürütülür.
Chain of Custody	Prosedürel Dokümantasyon	Delile kimin, ne zaman, nerede eriştiğinin tam günlüğü.	Delil zincirinin kırılması, vakanın düşmesi.
Procedural Compliance	Yasal Düzenlemeler (4. Değişiklik, GDPR vb.)	Arama emri sınırlarına veya KVKK kurallarına uyum.	Yasak meyve ağacı (Fruit of the poisonous tree) doktrini.

Adım 21: ISO/IEC 27037:2012 Standardının Operasyonel Rolü ISO/IEC 27037, potansiyel dijital delillerin tanımlanması (Identification), toplanması (Collection), elde edilmesi (Acquisition) ve korunmasına (Preservation) yönelik çerçeveyi çizen en üst uluslararası standarttır.¹⁸

Geliştirilen PE Analyzer aracı statik inceleme yapsa dahi, bu araca beslenen veri kaynakları (zararlı yazılımın RAM'den veya diskten alınan imajı), ISO/IEC 27037 prensiplerine uygun yetkili personel (DEFR - Digital Evidence First Responder) tarafından toplanmış olmalıdır.¹⁹ Standarda göre, kullanılan araçların ulusal yasalara uyumlu, metodolojisi açıklanabilir ve analizi gerçekleştiren uzmanların bu konularda sertifikaya edilmiş olması gerekmektedir.¹⁹

Adım 22: Statik Analiz Raporlarının Hukuki Niteliği ve "Attribution" (Fail Tespiti) Sınırı

Statik analiz, zararlı yazılımın çalıştırılmadan kodunun, import tablolarının ve yapısının incelenmesidir.²¹ Aracın Regex IoC modülü sayesinde tespit edilen C2 sunucu IP'leri veya "pefile" ile çıkarılan şüpheli API kümeleri, JSON raporu olarak dışa aktarıldığında bu raporlar *yalnızca olayın teknik anatomisini* kanıtlar.²¹ Bir saldırının teknik olarak "nasıl" ve "ne tür bir araçla" gerçekleştirildiğini göstermek hukuken yeterli değildir; soruşturmanın amacı "faili" (insanı veya kurumu) tespit etmektir (Attribution). Salt bir statik analiz raporu, klavyenin arkasındaki şahsın kimliğini ispatlayamaz.²¹ Bunun için İSS (İnternet Servis Sağlayıcı) kayıtları, güvenlik kamera görüntüleri veya ağ trafiği logları ile desteklenen bütünlük bir inceleme şarttır.

Adım 23: SHA-256 Otoritesi ve Kriptografik Bütünlük Araç tarafından zararlı yazılımın SHA-256 hash'inin çıkarılıp VirusTotal'e gönderilmesi işleminden çok daha önce, adli bilişim açısından kritik olan adım şudur: Olay anında diskten veya bellekten elde edilen fiziksel delilin

orijinal hash değeri ile, analiz platformuna yüklenen ve raporlanan dosyanın hash değeri *birebir aynı* (bit-for-bit match) olmalıdır.¹⁸ Eğer dosya analiz sistemine taşınırken kazara çift tıklanarak çalıştırılırsa veya meta verileri değiştirilirse, hash değeri bozulur. Hukuk nezdinde bütünlük (Integrity) kaybedilir ve elde edilen loC'ler şüpheli hale gelir.¹⁸

Adım 24: Kum Havuzu (Sandbox) İzolasyonunun Adli Önemi Aracın tüm bu işlemleri Docker tabanlı, geçici (ephemeral) ve izole bir Sandbox mimarisinde çalıştırması²³, yalnızca ana makineye zararlı bulaşma riskini sıfırlamakla kalmaz; aynı zamanda adli analiz ortamının steril (sterile environment) olduğunu kanıtlar. Dış ağ bağlantılarının kısıtlandığı veya sanal yönlendirmelerle kontrol edildiği bu ortamlar, çapraz bulaşma (cross-contamination) riskini elimine eder. Mahkemede karşı taraf uzmanının "Analiz yapılan makinede halihazırda var olan başka bir zararlı yazılım delile bulaşmış olabilir" argümanını çürütmek için bu izolasyon dokümantasyonu hayati önem taşır.

Adım 25: Federal Evidence Rule 901/902 ve Zaman Damgaları (Timestamps) ABD Federal Delil Kuralları 901 uyarınca delili sunan tarafın (proponent), delilin iddia edilen şey olduğunu ispat etmesi, 902 uyarınca da belirli dijital süreçlerin kendinden doğrulanabilir (self-authenticating) olması gerekir.¹⁸ Üretilen statik analiz JSON çıktılarının ve çıkarılan loC hash'lerinin güvenilir uluslararası zaman sunucularına (NTP) senkronize edilmiş ortamlar üzerinden nitelikli elektronik zaman damgaları (qualified electronic timestamp) ile damgalanması elzemdir.²⁵ Tıpkı salt bir "ekran görüntüsünün" mahkemede tek başına delil sayılmaması gibi (verinin SSL sertifikası, MHTML dökümü ve hash mühürleri ile ISO 27037'ye göre toplanması gerekir²⁵), analiz platformunun da her işlem adımını loglayıp mühürlemesi "Self-Authenticating" gereksinimlerini karşılar.

Adım 26: Delil Zincirinin (Chain of Custody) Dijital Ortamda Sürekliliği Chain of custody, delili kimin topladığını, nasıl paketleyip sakladığını, kimlerin eriştiğini, erişimlerin ne zaman ve ne amaçla gerçekleştiğini ve mevcut konumunu belgeleyen tam bir kronolojik kayıttır.¹⁸ Geliştirilen "PE Analyzer" platformunun, analiz komutunu veren uzmanın kimlik bilgilerini (Örn: Active Directory/IAM entegrasyonu), analiz edilen dosyanın geliş kaynağını ve analiz sonrasında dosyaların şifreli (AES-256) saklama alanlarına taşındığını detaylandıran otomatik bir "Chain of Custody Log" mekanizması barındırması hukuki savunulabilirliği maksimize eder.

Adım 27: Hassas Verilerin Korunması ve Hukuki Uyum Siber güvenlik incelemeleri genellikle kişisel veya kurumsal gizlilik içeren veri kümeleri üzerinde gerçekleştirilir. Zararlı yazılımın içerisine gömülü (hardcoded) halde bulunan e-posta adresleri veya dahili IP logları, soruşturma makamları ile paylaşılırken gizlilik standartlarına (Confidentiality) ve GDPR/KVKK gibi regülasyonlara uygun işlenmelidir.²⁴ Aracın veri çıkarma modülünün, kişisel verileri maskeleyen (data masking) veya yalnızca ilgili savcılık onayı ile açılabilir (key-escrow) bir mimaride çalışması gerekebilir.

Adım 28: Canlı Analiz (Live/Dynamic) ve Statik Analizin Hukuki Çarpışması Canlı analiz uçucu verileri (RAM, ağ bağlantıları) hedef alırken²⁶, statik analiz hareketsiz veri üzerinde gerçekleşir. Statik analiz, tekrarlanabilirlik ilkesi (repeatability) açısından mahkemelerde daha güçlü bir zemine sahiptir.²³ Zira statik analiz algoritması (örneğin Shannon Entropisi veya pefile parse işlemi), dosyaya tekrar tekrar uygulandığında her seferinde *aynı* sonucu verir. Oysa dinamik analizde (sandbox execution), zararlı yazılım C2 sunucusunun kapalı olduğunu

anladığında (environment checking) veya farklı bir zaman diliminde (time-bombed) çalıştırıldığında farklı davranışlar sergileyebilir. Bu stokastik durum, mahkemede "tutarsızlık" olarak değerlendirilebilir.

Adım 29: Ticari Adli Bilişim Araçlarına Karşı Özel (Custom) Araçlar Ticari araçlar, mahkeme sertifikasyonları, geniş dokümantasyonları ve teknik destek güvenceleri sayesinde adli süreçlerde varsayılan kabul edilir.²⁰ Üniversite veya kurum bünyesinde Python ile geliştirilen özel "PE Analyzer" gibi açık kaynaklı / özel kurgu yazılımların bilimsel geçerliliği, Daubert Standardı veya Frye Standardı kapsamında sınanabilir. Kullanılan algoritmaların (Entropi, Regex IoC, TF-IDF) akademik literatürde kabul görmüş, hata oranları bilinen ve akran denetiminden (peer-review) geçmiş yöntemler olması²⁰, aracın ürettiği verilerin mahkemede reddedilmesini engeller.

Adım 30: Adli Bilişim Hazırlığı (Forensic Readiness) Kurumlar, bir olay gerçekleşmeden önce dijital delilleri hukuka uygun olarak tespit etme, toplama ve koruma kapasitesini proaktif olarak inşa etmelidir. Bu kapasiteye "Adli Bilişim Hazırlığı" (Forensic Readiness) denir.¹⁹ "PE Analyzer" projesi, kurumsal ağda şüpheli bir dosya belirdiğinde anında izole edip statik anatomisini çıkararak, kurumun Forensic Readiness olgunluk seviyesini artıran stratejik bir otomasyon bileşeni olarak konumlandırılır.¹⁹

Bölüm V: Tehdit İstihbaratı ve Heuristik Risk Skorlama Motorunun Algoritmik Mimarisi

Dış kaynaklardan alınan tehdit istihbarat (Threat Intelligence) verilerinin, dosyanın yapısal

özellikleri ile harmanlanarak $0 - 100$ arası deterministik ve tekil bir skor üretmesi, istatistiksel zafiyetler barındırır. Bu zafiyetleri gidermek için çok boyutlu bir Heuristik Skorlama Algoritması kurgulanmalıdır.

Adım 31: False Positive (Yanlış Pozitif) Çıkmazı ve VirusTotal Gerçeği Aracın VirusTotal v3 API'si entegrasyonu ile canlı tehdit istihbaratı alması kritik bir katmandır. Ancak VirusTotal skorları hiçbir zaman mutlak doğru (ground truth) olarak değerlendirilemez (False Positives).²⁸ Özellikle oyunların Anti-Cheat motorları, lisans koruma yazılımları veya UPX ile sıkıştırılmış meşru kurumsal iç yazılımlar, güvenlik satıcılarının (vendor) statik imzalarına ve entropi eşiklerine takılarak "malicious" (zararlı) olarak etiketlenebilir.³ Bu durum, ham API yanıtlarının doğrudan nihai karara yansıtılmasının büyük bir hata olduğunu gösterir.

Adım 32: Satıcı İtibarı ve Ağırlıklandırılmış Konsensüs (Weighted Vendor Consensus)

Risk skorlama motoru, 70 farklı antivirüs motorunun verdiği "Zararlı" yanıtını salt bir yüzde hesabına (örneğin $14/70 = \%20$) dökmek yerine "Ağırlıklandırılmış Konsensüs" (Weighted Consensus) algoritması kullanmalıdır. ESET, Kaspersky, Microsoft veya CrowdStrike gibi gelişmiş sezgisel (heuristic) analiz ve pazar payına sahip "Tier-1" vendor'ların tespitleri (

$w = 0.85$) ile, sadece eski imza veritabanlarına dayanan bilinmeyen vendor'ların tespitleri ($w = 0.15$) farklı katsayılarla çarpılmalıdır.

$$Score_{VT} = \frac{\sum_{i=1}^n (Vendor_Weight_i \times Detection_Status_i)}{\sum Vendor_Weight_i}$$

Adım 33: TF-IDF (Term Frequency - Inverse Document Frequency) ile API Vektörizasyonu

Geliştirilen araçtaki statik analiz motorunun bulduğu "şüpheli API çağrıları" doğrudan sayısal bir "şüphe" metriğine çevrilemez. Her API aynı tehlike oranına sahip değildir. Bu aşamada Doğal Dil İşleme (NLP) alanı olan Term Frequency-Inverse Document Frequency (TF-IDF) yöntemi, API dizilerini vektörize etmek için entegre edilmelidir.³¹ TF-IDF algoritması, bir API'nin ilgili zararlı dosya içerisindeki bulunma/çağırılma sıklığı (Term Frequency) ile, evrensel iyi/kötü yazılım veritabanındaki nadir bulunma oranını (Inverse Document Frequency) çarparak benzersiz bir

ağırlık (W_{API}) hesaplar.³¹

Adım 34: API Ağırlıklandırma Dinamikleri GetProcAddress veya LoadLibraryA gibi fonksiyonlar hem zararlı yazılımların paket açma işlemlerinde hem de tamamen meşru

yazılımların dinamik modül yüklemelerinde kullanıldığı için⁸, bu API'lerin IDF skoru evrensel kümelerde düşüktür. Buna karşılık, bellek enjeksiyonu için kullanılan VirtualAllocEx, WriteProcessMemory veya EDR kancalarını bypass etmek için kullanılan

NtProtectVirtualMemory gibi API'lerin birlikte görülme (co-occurrence) IDF değerleri muazzam derecede yüksektir.³¹

Örnek API Çağrısı	Meşru Kullanım (TF Sıklığı)	Zararlı Kullanım (TF Sıklığı)	IDF Skoru (Nadirliği)	Ağırlık (Risk) Etkisi
GetProcAddresss	Yüksek	Yüksek	Çok Düşük (Sıradan)	Minör (Tek başına risksiz)
LoadLibraryA	Yüksek	Yüksek	Çok Düşük (Sıradan)	Minör (Tek başına risksiz)
VirtualAllocEx	Çok Düşük	Yüksek	Yüksek (Nadir)	Majör (İşlem içi tahsis)
CreateRemote Thread	Yok Denecek Kadar Az	Yüksek	Çok Yüksek (Çok Nadir)	Kritik (Kesin Enjeksiyon)

Adım 35: Entropi Değerinin Normalizasyonu ve Lineer Olmayan Cezalandırma (Penalty)

Entropi skoru (0.0 - 8.0 arası), risk motoruna lineer bir çarpan olarak eklenmemelidir. Bölüm I'de

tartışılan düşük entropili zararlıların varlığı nedeniyle ¹, entropinin yüksek olması risk skorunu radikal şekilde artırırken (pozitif çarpan), entropinin düşük olması risk skorunu radikal şekilde düşürmemelidir. Algoritma, $Entropy > 7.0$ ise genel skora sabit bir $+15$ puan eklemeli,

ancak $Entropy < 5.0$ ise herhangi bir puan düşme (ödüllendirme) işlemi yapmamalıdır.

Adım 36: Hibrit Topluluk Karar Motoru (Hybrid Ensemble Model) Sistem, bireysel makine öğrenmesi algoritmalarının yanılma payını düşürmek için Ensemble (Topluluk) modelleme kullanılmalıdır.³³ Rastgele Orman (RF) algoritması ile hesaplanan özellik alt kümeleri, Derin Sinir Ağları (DNN) gibi modellerle birleştirilerek, veri kümesi özelliklerinin içsel karakteristikleri yakalanır.³⁶ MR-ANNIGMA (Maximum Relevance - Artificial Neural Net Input Gain Measurement Approximation) benzeri süzgeçleme (wrapper-filter) yöntemleri ile gereksiz ve gürültülü API/Entropi özellikleri dataset'ten elenerek modelin tahmin süresi (execution time) ve doğruluğu optimize edilir.³³

Adım 37: Bulanık Mantık (Fuzzy Logic) Entegrasyonu Kesin eşik değerlerinin ("Entropi 7.1 ise zararlıdır, 6.9 ise temizdir") yarattığı Boolean çelişkisini çözmek için Mamdani Bulanık Çıkarım Sistemi (Fuzzy Inference System) kullanılmalıdır.³⁷ Sisteme beslenen girdiler (Entropi, API ağırlığı, VT Skoru); "Düşük, Orta, Yüksek, Kritik" gibi dilsel (linguistic) değişkenlere dönüştürülür. Uzman kurallar tabanı (Rule-Base) kullanılarak (Örn: "EĞER Entropi Yüksek VE API Ağırlığı Kritik İSE Risk=Kritik") bir risk matrisi çıkarılır ve ardından "Durulanma"

(Defuzzification) işlemi ile $0 - 100$ arası kesin bir rakama indirgenir.³⁷

Adım 38: Matematiksel Nihai Risk Skoru Formülü

Sistemin farklı modüllerden topladığı verileri entegre edeceği nihai Heuristik Risk Puanı hesaplaması, esnek ağırlıklandırılmış bir denklem olmalıdır:

$$Global_Risk_Score = (\alpha \cdot \Psi(Entropy)) + (\beta \cdot \Phi(API_{TFIDF})) + (\gamma \cdot Score_{VT}) + (\delta \cdot \Omega(IoC_{Regex}))$$

Bu denklemde yer alan α, β, γ ve δ katsayıları, her bir faktörün önem derecesini belirleyen hiperparametrelerdir. ($\alpha + \beta + \gamma + \delta = 1.0$). $\Omega(IoC_{Regex})$, C2 IP/URL adreslerinin bilinen tehdit veritabanlarındaki risk seviyesine göre üretilen logaritmik veya üstel bir metriği ifade eder.

Adım 39: Dinamik Ağırlık Adaptasyonu (Dynamic Weight Shifting) Algoritma statik parametrelerle sınırlı kalmamalıdır. Örneğin, "Zero-Day" bir zararlı incelendiğinde VirusTotal

Skoru 0 dönebilir ($Score_{VT} = 0$). Eğer sistem, API diziliminde VirtualAlloc -> WriteProcessMemory -> CreateRemoteThread sekansını yakalamışsa (Aşırı yüksek TF-IDF ağırlığı), algoritma VT'nin sıfır dönmesini göz ardı etmeli ve β parametresinin (API analizinin) ağırlığını dinamik olarak artırıp γ (VT Skoru) ağırlığını düşürerek "Process Injection" zafiyeti

şüphesiyle proaktif yüksek risk puanı üretmelidir.²⁹

Adım 40: Kuantum-Klasik Füzyon (Quantum-Classical Fusion) Modelleri Perspektifi

Gelecekteki heuristik skorlama motorları, klasik makine öğrenimi modellerinin ötesine geçerek davranışsal özellikleri (örneğin API kullanım grafiklerini ve yapısal-entropi biçimciklerini) kuantum özellik haritalarına (quantum feature maps) kodlayan genlik kodlaması (amplitude encoding) ve Kuantum Fourier Dönüşümü (QFT) gibi yöntemlere evrilmektedir.³⁵ Bu karma sistemler (hybrid analysis), yanlış pozitifleri (false positives) en düşük seviyeye indirerek tehdit istihbaratını kuantum düzeyinde hızlandırır.³⁵

Adım 41: Büyük Dil Modelleri (LLM) ve Zero-Label Açık Dünya Sınıflandırması Açık dünya senaryolarında, etiketlenmiş verilere (labeled datasets) bağımlı makine öğrenmesi sistemleri tıkanmaktadır. Önceden eğitilmiş ağırlıklı Büyük Dil Modelleri (LLMs) kullanılarak zararlı yazılım bayt-kodlarından ve assembly talimatlarından "Doğal Dil" formatında çıkarımlar yapmak (SBAN - Source-Binary-Assembly-Natural Language yaklaşımı)³⁴, sıfır-etiketli (zero-label) analiz imkanı sunar. ModernBERT stili Byte-Pair Encoding (BPE) kodlayıcılar³⁸, çoklu-bayt kod desenlerini dil tokenleri gibi işleyerek³⁸, VirusTotal istihbaratı olmaksızın kod düzeyinde semantik anomaliyi keşfedebilir.

Adım 42: Tehdit Modeli Risk Çıktısının Olay Müdahale Ekipleri (SOC) İçin

Kategorizasyonu Hesaplanan $0 - 100$ arası sürekli (continuous) değer, SOC (Security Operations Center) analistleri için eyleme geçirilebilir (actionable) bir göstergeye dönüşmelidir.²⁷ Örneğin;

- **0-25 Puan (Benign/Temiz):** Düzenli işlem. Sistem engellemesi yapılmaz.
- **26-55 Puan (Suspicious/Şüpheli):** Dinamik Kum Havuzu (Sandbox) analizine yönlendirilir, ağ aktiviteleri kısıtlanır.
- **56-85 Puan (High Risk/Yüksek Risk):** Uç nokta güvenlik cihazları uyarılır, cihazın kurum ağıyla bağlantısı (isolation) askıya alınır.
- **86-100 Puan (Malicious/Zararlı Kritik):** Vaka müdahale (Incident Response) protokolü başlar, adli bilişim zinciri (Chain of Custody) için bellek imajı (memory dump) alınır.

Adım 43: MITRE ATT&CK Çerçevesi İle Haritalandırma (Mapping) Aracın ürettiği Heuristik Motor Raporu, sadece bir risk skoru sunmanın ötesinde, tespit ettiği taktik ve teknikleri MITRE ATT&CK matrisiyle haritalandırılmalıdır.⁵ API Hashing tespiti yapıldıysa bunu doğrudan T1027.007 (Obfuscated Files or Information: Dynamic API Resolution) kodu ile ilişkilendirmesi, tehdit avcılarına (Threat Hunters) davranışsal reaktif stratejiler üretmelerinde yardımcı olur.⁵

Adım 44: Hata Dağılımını Azaltan "Erken Füzyon" (Early Fusion) Yaklaşımı Araç içerisindeki modüllerin verileri analiz sonunda değil, veri işleme aşamasında harmanlanmalıdır. El yapımı özellikler (PE-histogramları, API kullanımı, entropi sinyalleri) ile derin öğrenme özellikleri (CNN-tabanlı n-gramlar, gri tonlamalı dokular) XGBoost gibi ağaç topluluklarına (tree ensembles) erken füzyon tekniği ile birlikte sokularak karar destek motorunun genelleme kapasitesi artırılmalıdır.³⁵

Adım 45: Yasal Çerçeve ve Heuristik Tespitte Karar Açıklanabilirliği (XAI - Explainable AI)

Üretilen $0 - 100$ skorunun arkasındaki matematiksel süreç, bir adli soruşturma sırasında

"Kara Kutu" (Black Box) olarak nitelendirilmemelidir. Explainable AI (Açıklanabilir Yapay Zeka) mimarisi kullanılarak, SHAP veya LIME gibi algoritmalarla skorun %40'ının entropiden, %35'inin spesifik olarak WriteProcessMemory API'sinden ve %25'inin IoC eşleşmesinden geldiği şeffaf bir şekilde raporlanmalı, hukuki süreçteki ISO 27037 "metodolojik açıklanabilirlik" beklentisine uyum sağlanmalıdır.¹⁹

Bölüm VI: Araştırma Notları (Research Notes) Şablonu

Araştırma ve sistem dizayn sürecinde karşılaşılan bazı teknik çıkmazlar ve akademik çözüm bekleyen sorular aşağıda dokümente edilmiştir.

Çıkmaz Sokaklar (Dead Ends)

Adım 46: Dead End 1 - Statik Entropinin Güvenilirliğinin Çöküşü (Absolut Limit) Sistem kurgusunda Shannon Entropisi'nin ana paketleyici tespiti (packer detection) mekanizması olarak kullanılması, ampirik deneyler sonucu bir çıkmaz sokak (dead end) olarak tescillenmiştir. Düşük entropili (Low-Entropy) paketleme kullanan zararlı yazılımların %30'luk bir ekosistem

yaygınlığına ulaşması ¹, ^{7.0} ve üzeri threshold'ların güvenlik stratejisindeki merkeziliğini yok etmiştir. Matematiksel olarak kodlanmış zararlı yazılım bölümlerinin bayt dolguları (byte padding)

ve istatistiksel manipülasyonlarla entropi değerini meşru ^{4.5 – 5.5} bandına indirebilmesi, statik Shannon Entropisi analizinin sınırını (Absolute Limit) temsil etmektedir.³

Adım 47: Dead End 2 - EAT Ayırıştırmasında Statik API Avının Yetersizliği Geliştirilen araçtaki statik analiz motorunun yalnızca dosyanın disk üzerindeki IMAGE_IMPORT_DESCRIPTOR yapılarını okumaya programlanmış olması kritik bir zafiyettir. API Hashing teknikleriyle birleştirilen PEB Yürüyüşü (PEB Walk) stratejileri, zararlı yazılımın disk üzerindeki IAT içerisinde "LoadLibrary" veya "GetProcAddress" dahil hiçbir işletim sistemi API'si izi bırakmamasına yol açmaktadır.⁸ Eğer kötü amaçlı bir dosya çalışma zamanında (runtime) bellekteki Export Address Table (EAT) tablosunu kendi Assembly yönergeleri ile ayırıştırırsa, statik analiz aracının Regex veya Pefile ayırıştırıcıları ile bu dinamik yapıyı tespit etme imkanı teorik ve pratik olarak sıfırdır.

Kalan Sorular (Questions Remaining)

Adım 48: Kalan Sorular 1 - Adli Bilişim & Makine Öğrenmesi Çelişkisi Aracın Heuristik Risk Skorlama motorunda kullanılacak Derin Öğrenme (Deep Learning) veya XGBoost tabanlı çok boyutlu ağırlıklandırma algoritmaları doğası gereği "Black-Box" (Kara Kutu) yapısındadır. Karar ağırlıklarının ve nöral ağ katmanlarının bir insan (analist) tarafından adım adım, deterministik olarak doğrulanamadığı bu modellerin ürettiği raporlar, çapraz sorgu (cross-examination) aşamasında avukatlar tarafından "Çalışma mekanizması belirsiz ve manipülasyona açık bir makine tahmini" olarak nitelendirildiğinde; bu çıktıların ISO/IEC 27037 standardının "tekrarlanabilirlik ve bilimsel açıklanabilirlik" ilkelerine ¹⁹ hukuken nasıl entegre edilebileceği belirsizliğini korumaktadır.

Adım 49: Kalan Sorular 2 - Bulut Tabanlı İzolasyon Standardının Güvenilirliği Docker tabanlı izole Kum Havuzu (Sandbox) mimarisi, geleneksel host sistemi kontaminasyonunu

engellese de; Container Escape (Konteynerden Kaçış) zafiyetlerini kullanan, doğrudan Linux Çekirdeği modüllerine (LKM) sızan veya Windows mimarisinde Ring 0 (Kernel-Mode) ayrıcalıklarına ulaşan Direct Syscall tabanlı ¹³ gelişmiş tehditler incelenirken, bu izolasyonun garantisi var mıdır? Adli bilişim zincirindeki bütünlüğü (Integrity) ¹⁸ tartışmasız hale getirmek için konteynerizasyon tabanlı hafif izolasyonlar yerine tam teşekküllü (Bare-Metal) donanım izolasyonlarına geçiş maliyet/performans ve yasal gereklilik açısından zorunlu mudur?

Adım 50: Kalan Sorular 3 - Kuantum Karar Motorlarının Ölçeklenebilirliği Matematiksel risk skorum ve yanlış pozitif oranlarını düşürme süreçlerinde geleneksel TF-IDF yerine, API çağrıları arasındaki örtük ilişkileri Kuantum Fourier Dönüşümü (QFT) genliklerine kodlayan "Kuantum-Klasik Füzyon Modellerine" (Quantum-Classical Fusion) ³⁵ teorik geçiş, donanım kaynaklarının mevcut sınırları dahilinde ticari ölçeklenebilirliğe sahip midir? Önümüzdeki on yıl içerisinde bu mimarinin günlük SOC operasyonlarında kullanılabilmesi için donanım iterasyonları ne hızda ilerlemelidir?

Alıntılanan çalışmalar

1. Prevalence and Impact of Low-Entropy Packing Schemes in the Malware Ecosystem, erişim tarihi Haziran 2, 2026, <https://www.ndss-symposium.org/ndss-paper/prevalence-and-impact-of-low-entropy-packing-schemes-in-the-malware-ecosystem/>
2. Prevalence and Impact of Low-Entropy Packing Schemes in the Malware Ecosystem, erişim tarihi Haziran 2, 2026, <https://www.semanticscholar.org/paper/Prevalence-and-Impact-of-Low-Entropy-Packing-in-the-Mantovani-Aonzo/05462beae637fcbca321174622f0cefc4623b85>
3. Prevalence and Impact of Low-Entropy Packing Schemes in the Malware Ecosystem - Semantic Scholar, erişim tarihi Haziran 2, 2026, <https://pdfs.semanticscholar.org/2538/14d8fc68fb95da05e7c40ba642764d16a458.pdf>
4. Exotic Compilers as a Malware Evasion Technique (Discussion Paper) - CEUR-WS.org, erişim tarihi Haziran 2, 2026, <https://ceur-ws.org/Vol-2400/paper-35.pdf>
5. Hunting Polymorphic Malware. Methods, Hypotheses, and Detection... | by Deniz Topaloglu, erişim tarihi Haziran 2, 2026, <https://medium.com/@d3lt4labs/hunting-polymorphic-malware-5d687194f19a>
6. Adversarially Robust Assembly Language Model for Packed Executables Detection - arXiv, erişim tarihi Haziran 2, 2026, <https://arxiv.org/html/2509.15499v1>
7. Enhancing Malware Detection via RGB Assembly Visualization and Hybrid Deep Learning Models - MDPI, erişim tarihi Haziran 2, 2026, <https://www.mdpi.com/2076-3417/15/13/7163>
8. Obfuscated Files or Information: Dynamic API Resolution, Sub-technique T1027.007, erişim tarihi Haziran 2, 2026, <https://attack.mitre.org/techniques/T1027/007/>
9. API Hashing — Why Malware Loves (And You Should Care) | by ..., erişim tarihi Haziran 2, 2026,

<https://securitymaven.medium.com/api-hashing-why-malware-loves-and-you-should-care-77c5135d9aaa>

10. PEB Walk: Avoid API calls inspection in IAT by analyst and bypass static detection of AV/EDR | by Cytomate, erişim tarihi Haziran 2, 2026, <https://cytomate.medium.com/peb-walk-avoid-api-calls-inspection-in-iat-by-analyst-and-bypass-static-detection-of-av-edr-ee7b0dd9c33c>
11. Offensive-Panda/PEB_WALK_AND_API_OBFUSCATION_INJECTION: This exploit use PEB walk technique to resolve API calls dynamically, obfuscate all API calls to perform process injection. - GitHub, erişim tarihi Haziran 2, 2026, https://github.com/Offensive-Panda/PEB_WALK_AND_API_OBFUSCATION_INJECTION
12. Dive into PEB Walk in Malware Analysis - root@fareed:~#, erişim tarihi Haziran 2, 2026, <https://fareedfauzi.github.io/2024/07/13/PEB-Walk.html>
13. EDR evasion: techniques, real-world breaches, and defenses - Vectra AI, erişim tarihi Haziran 2, 2026, <https://www.vectra.ai/topics/edr-evasion>
14. Antivirus and EDR Bypass Techniques Explained - Vaadata, erişim tarihi Haziran 2, 2026, <https://www.vaadata.com/en/blog/antivirus-and-edr-bypass-techniques/>
15. Stealth Syscall Execution: Bypassing ETW, Sysmon, and EDR Detection, erişim tarihi Haziran 2, 2026, <https://www.darkrelay.com/post/stealth-syscall-execution-bypass-edr-detection>
16. LayeredSyscall - Abusing VEH to Bypass EDRs | White Knight Labs, erişim tarihi Haziran 2, 2026, <https://whiteknightlabs.com/2024/07/31/layeredsyscall-abusing-veh-to-bypass-edr-s/>
17. Just Carry A Ladder: – Why Your EDR Let Pikabot Jump Through - VMRay, erişim tarihi Haziran 2, 2026, <https://www.vmrays.com/just-carry-a-ladder-why-your-edr-let-pikabot-jump-through/>
18. Compliance for Evidence: Key Consideration and Best Practices, erişim tarihi Haziran 2, 2026, <https://digitalevidence.ai/blog/compliance-for-evidence>
19. international standard iso/iec 27037, erişim tarihi Haziran 2, 2026, <https://amnafzar.net/files/1/ISO%2027000/ISO%20IEC%2027037-2012.pdf>
20. The admissibility of digital evidence from open-source forensic tools: Development of a framework for legal acceptance - PMC, erişim tarihi Haziran 2, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12431127/>
21. Deep Dive. DFIR: Digital Forensics & Incident... | by Or Ben Ari | Medium, erişim tarihi Haziran 2, 2026, <https://medium.com/@OrBenAri/dfir-deep-dive-3b1c8d355e0d>
22. Common Types of Digital Forensics in the Modern Courtroom, erişim tarihi Haziran 2, 2026, <https://guardian-forensics.com/digital-forensics/complete-guide-to-digital-forensics/>
23. Mastering Malware Analysis: Types, Stages, and Best Practices to Stay Protected, erişim tarihi Haziran 2, 2026, <https://www.scarlettgroup.com/blog/malware-analysis-explained>
24. Malware Forensics: Investigating & Analysis | Vaia, erişim tarihi Haziran 2, 2026, <https://www.vaia.com/en-us/explanations/law/forensic-science/malware-forensics/>

25. Web Evidence Acquisition: ISO 27037 Forensic Guide - TrueScreen, erişim tarihi Haziran 2, 2026, <https://truescreen.io/articles/web-evidence-acquisition-iso-27037-guide/>
26. Cybersecurity Forensics: Types and Best Practices - SentinelOne, erişim tarihi Haziran 2, 2026, <https://www.sentinelone.com/cybersecurity-101/cybersecurity/cybersecurity-forensics/>
27. US10404737B1 - Method for the continuous calculation of a cyber security risk index - Google Patents, erişim tarihi Haziran 2, 2026, <https://patents.google.com/patent/US10404737B1/en>
28. System API Vectorization for Malware Detection - IEEE Xplore, erişim tarihi Haziran 2, 2026, <https://ieeexplore.ieee.org/iel7/6287639/10005208/10124968.pdf>
29. Optimal Weighted Voting-Based Collaborated Malware Detection for Zero-Day Malware: A Case Study on VirusTotal and MalwareBazaar - MDPI, erişim tarihi Haziran 2, 2026, <https://www.mdpi.com/1999-5903/16/8/259>
30. Prevalence and Impact of Low-Entropy Packing Schemes in the ..., erişim tarihi Haziran 2, 2026, <https://www.ndss-symposium.org/wp-content/uploads/2020/02/24297.pdf>
31. Channel Features and API Frequency-Based Transformer Model for Malware Identification, erişim tarihi Haziran 2, 2026, <https://www.mdpi.com/1424-8220/24/2/580>
32. A Kullback-Liebler divergence-based representation algorithm for malware detection - PMC, erişim tarihi Haziran 2, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10557483/>
33. Machine learning techniques for imbalanced multiclass malware classification through adaptive feature selection - PMC, erişim tarihi Haziran 2, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12190268/>
34. Automated Malware Family Classification using Weighted Hierarchical Ensembles of Large Language Models - arXiv, erişim tarihi Haziran 2, 2026, <https://arxiv.org/html/2604.02490>
35. Hybrid Analysis for Malware Detection - Emergent Mind, erişim tarihi Haziran 2, 2026, <https://www.emergentmind.com/topics/hybrid-analysis-for-malware-detection>
36. A Hybrid Wrapper-Filter Approach for Malware Detection, erişim tarihi Haziran 2, 2026, <https://openresearch-repository.anu.edu.au/server/api/core/bitstreams/5dc3dfd6-0a9a-45b9-80da-50cd1e2d7251/content>
37. Cyber-Attacks Risk Analysis Method for Different Levels of Automation of Mining Processes in Mines Based on Fuzzy Theory Use - PMC, erişim tarihi Haziran 2, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC7766562/>
38. MalwarePT: A Binary-Level Foundation Model for Malware Analysis - arXiv, erişim tarihi Haziran 2, 2026, <https://arxiv.org/html/2605.16455v1>